Evaluating ChatGPT5 on HW5's code problems gives me the following observations and below listed my comment on the whole chat log.

Core Observations:

1. ChatGPT has wonderful code completion ability even its prompt is only based on part of the whole JupyterNotebook. Almost all the generated codes are correct at the first time and can easily run on the colab homework.

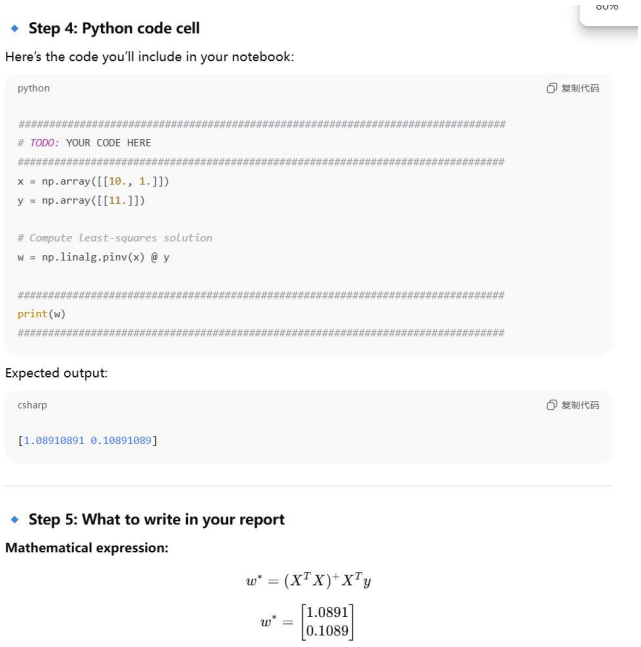
2. ChatGPT also have great debug ability simply with the hints from the error information of the code cell's output. There was a time when it generated a singular matrix and tried to inverse it. And it quickly fixed the problem by modifying the line causing the matrix to be singular.

3. ChatGPT can even answer some questions which requires the result from the code cell, though it can't actually run it. In other words, it generates the correct answer which should be output by the correctly implemented code cell. As an LLM, this is amazing.

4. However, when I keep testing the ability listed above, GPT began to make some mistakes especially when the question is related to a specific number. I think GPT can derive some simple code cell's output from a mathematical way, like solving a written problem. But can not handle complex codes.

5. For the code analysis questions, I provided GPT with some curve images which are necessary. And GPT can derive good intuition from the results such as the loss curve.

Comments:



Step 4: Python code cell

Here's the code you'll include in your notebook:

```python
################################################################################
# TODO: YOUR CODE HERE
################################################################################
x = np.array([[10., 1.]])
y = np.array([[11.]])

# Compute least-squares solution
w = np.linalg.pinv(x) @ y

################################################################################
print(w)
################################################################################
```

Expected output:

```
[1.08910891 0.10891089]
```

Step 5: What to write in your report

Mathematical expression:

$$w^* = (X^T X)^+ X^T y$$

$$w^* = \begin{bmatrix} 1.0891 \\ 0.1089 \end{bmatrix}$$

GPT generates correct implementation of code and gives the expected output accurately, with some mathematical inference step.

Also, this problem is the simplest one and GPT can derive the real output by "doing" math.

Excellent — this part of the question checks that **gradient descent** (using PyTorch) actually recovers the same solution we derived analytically in the previous section. Let's go step by step again so you can fill in your notebook and report confidently.

🔹 Step 1: Define the simple linear model

We're solving $y = Xw$ where $X = [[10, 1]]$ and $y = [[11]]$.
So we just need a linear layer with **2 inputs → 1 output**, and **no bias** (since our equation didn't include a bias term).

```python
复制代码

#########################################################################
# TODO: YOUR CODE HERE
#########################################################################
net = nn.Linear(2, 1, bias=False)
#########################################################################
```

🔹 Step 2: Initialize the weights to zero

We can then zero-initialize the weights before training:

```python
复制代码

# Initialize weights with 0
net.load_state_dict({k: v * 0 for k, v in net.state_dict().items()})
```

🔹 Step 4: Explanation (for your report)

Output (text-only):

```lua
复制代码

x = [[20.  2.]
 [ 0.  2.]
 [20.  0.]
 [ 0.  0.]]
y = [[11.]
 [11.]
 [11.]
 [11.]]
w = [[0.55]
 [2.75]]
```

Interpretation:

The dropout-augmented least-squares solution is:

$$w^*_{\text{dropout}} = [0.55,\ 2.75]$$

Would you like me to continue to the **"Dropout, Gradient Descent"** part next (where we implement dropout layers in PyTorch and compare training results)?

与Claude比较

GPT tends to guide you step-by-step. And solve the problem in a well-organized way.

Also, it will provide a final version of the complete notebook cell.

Wrong answer here. The expected x and y provided by GPT are correct. But the w is far from the right answer w = [[0.36666667] [3.66666667]]

Perfect. Although your calculation on w is not correct, but your code actually outputs the right answer, which is:
x = [[20 2] [20 0] [0 2] [0 0]]
y = [[11] [11] [11] [11]]
w = [[0.36666667] [3.66666667]].
Don't worry if you answer these kind of questions wrongly when the question is about to provide the output of actual code. Since you can't actually run the code as an LLM, this kind of mistake is totally understandable, as long as your code is right and can generate correct answer.

Now comes to the next sub-question. This part is about Dropout, Gradient Descent. Your job is still complete the code and answer the question:

Give feedback about the answer it generated. And make it focus on generating better code instead of guessing the output of the cell.

🔹 4. Final written answer

Loss curve:
With a large batch size (1024), the loss decreases rapidly at first and then follows a smooth, gentle downward slope. Unlike the previous experiment (batch = 1), the curve is not spiky or noisy.

Reason for difference:
In the small-batch setting, each update sees a different random dropout mask, leading to highly variable gradients and a noisy loss curve. A large batch averages across many dropout masks, reducing gradient variance and producing a stable, smooth learning trajectory.

Weights comparison:
The trained weights (≈ [0.3752, 3.4848]) are very close to the analytic dropout least-squares solution ([0.3667, 3.6667]), confirming that larger batches help the stochastic gradient descent approximate the expected least-squares solution more accurately.

Good answer on written questions, based on the code and cell's output.

```
/usr/local/lib/python3.12/dist-packages/numpy/linalg/_linalg.py
in _raise_linalgerror_singular(err, flag)
    102
    103 def _raise_linalgerror_singular(err, flag):
--> 104     raise LinAlgError("Singular matrix")
    105
    106 def _raise_linalgerror_nonposdef(err, flag):

LinAlgError: Singular matrix
"
```

Please try to solve this problem.

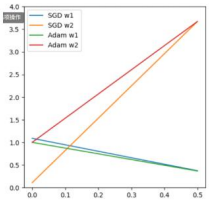
Excellent — this is a really good debugging question.

The **"Singular matrix"** error happens because for some dropout rates p, the weighted design matrix $X^T S X$ becomes **non-invertible**. That occurs mainly when dropout probability p approaches 0 or 1, since then some rows of X (or their weights) become zero or linearly dependent.

Let's analyze and then fix this safely.

Code with bugs! Not check the special case that will make the matrix a singular one.

Quickly solved by one simple hint given by the cell's output. I don't even need to manually guide it.



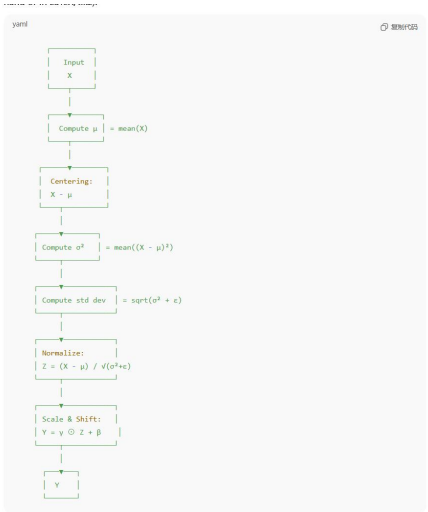
Great. Your code can generate the right curve. And your written answer is almost right. But the most intuitive answer is:"When dropout rates are positive, there is a unique loss-minimizing solution. Achieving it involves making w2 larger than w1. When dropout rates are zero, there are many loss-minimizing solutions, and the network chooses the norm-minimizing one, which makes w1 larger than w2."

Me providing the GPT with result curve image to let it finish the analyzing question.

```
Train Epoch: 1 of 10 Train Loss: 2.265, Val Loss: 2.236255, Val Accuracy: 21.420000
Train Epoch: 2 of 10 Train Loss: 2.148, Val Loss: 2.141020, Val Accuracy: 26.870001
Train Epoch: 3 of 10 Train Loss: 2.037, Val Loss: 2.044776, Val Accuracy: 33.549999
Train Epoch: 4 of 10 Train Loss: 1.877, Val Loss: 1.905086, Val Accuracy: 41.910000
Train Epoch: 5 of 10 Train Loss: 1.721, Val Loss: 1.749413, Val Accuracy: 47.990002
Train Epoch: 6 of 10 Train Loss: 1.557, Val Loss: 1.533527, Val Accuracy: 57.430000
Train Epoch: 7 of 10 Train Loss: 1.352, Val Loss: 1.271250, Val Accuracy: 70.279999
Train Epoch: 8 of 10 Train Loss: 1.218, Val Loss: 1.074222, Val Accuracy: 79.620003
Train Epoch: 9 of 10 Train Loss: 1.061, Val Loss: 0.893338, Val Accuracy: 83.779999
Train Epoch: 10 of 10 Train Loss: 1.010, Val Loss: 0.764200, Val Accuracy: 87.860001

Accuracy on clean data: 2205/10000 (22%)
Accuracy on data with cheating feature: 8786/10000 (88%)"
```

Me providing the GPT with training log to let it finish the analyzing question.



Even tried to generate the structure of reliance between different variables to answering the question.
The graph is basically correct too.